

P3054R0

2023-12 Library Evolution Poll Outcomes

Published Proposal, 2024-01-13

Authors:

[Inbal Levi - Library Evolution Chair](#) (MPGC Services LTD)

[Fabio Fracassi - Library Evolution Assistant Chair](#) (CODE University of Applied Sciences)

[Ben Craig - Library Evolution Assistant Chair](#) (Raven)

[Billy Baker - Library Evolution Incubator Chair](#) (NVIDIA)

[Nevin Liber - Library Evolution Incubator Assistant Chair and Admin Chair](#) (Argonne National Laboratory)

[Corentin Jabot - Library Mailing List Review Manager](#)

Source:

[GitHub](#)

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Audience:

WG21

Table of Contents

1 Introduction

2 Poll Outcomes

3 Selected Poll Comments

3.1 Poll 1: Send "[P0876R14]: fiber_context - fibers without scheduler" to Library Working Group for C++26.

3.2 Poll 2: Send "[P0447R26]: Introduction of std::hive to the standard library" to Library Working Group for C++26.

3.3 Poll 3: Send "[P2542R7]: views::concat" to Library Working Group for C++26.

3.4 Poll 4: Send "[P2642R5]: Padded mdspan layouts" to Library Working Group for C++26.

- 3.5 Poll 5: Send "[P2663R5]: Proposal to support interleaved complex values in std::simd" to Library Working Group for C++26.
- 3.6 Poll 6: Send "[P2810R2]: is_debugger_present is_replaceable" to Library Working Group for C++26.
- 3.7 Poll 7: Send "[P2809R2]: Trivial infinite loops are not Undefined Behavior" to Library Working Group for C++26.
- 3.8 Poll 8: Send "[P2845R5]: Formatting of std::filesystem::path" to Library Working Group for C++26.
- 3.9 Poll 9: Send "[P2862R1]: text_encoding::name() should never return null values" to Library Working Group for C++26.
- 3.10 Poll 10: Send "[P2867R1]: Remove Deprecated strtreams From C++26" to Library Working Group for C++26.
- 3.11 Poll 11: Send "[P2869R3]: Remove Deprecated shared_ptr Atomic Access APIs From C++26" to Library Working Group for C++26.
- 3.12 Poll 12: Send "[P2866R1]: Remove Deprecated Volatile Features From C++26" to Library Working Group for C++26.
- 3.13 Poll 13: Send "[P2944R2]: Comparisons for reference_wrapper" to Library Working Group for C++26.
- 3.14 Poll 14: Send "[P2933R1]: Extend header function with overloads for std::simd" to Library Working Group for C++26.
- 3.15 Poll 15: Send "[P2976R0]: Freestanding Library: algorithm, numeric, and random" to Library Working Group for C++26.
- 3.16 Poll 16: Send "[P2968R2]: Make std::ignore a first-class object" to Library Working Group for C++26.
- 3.17 Poll 17: Send "P2999R3: Sender Algorithm Customization" to Library Working Group for C++26.
- 3.18 Poll 18: Approve "[P2267R1]: Library Evolution Policies" and create SD-9 as described.
- 3.19 Poll 19: Approve "[P2760R1]: A Plan for C++26 Ranges".

References

Informative References

§ 1. Introduction

In 2023-12, the C++ Library Evolution group conducted a series of electronic decision polls [\[P3053R0\]](#). This paper provides the results of those polls and summarizes the results.

In total, 24 people participated in the polls. Some participants opted to not vote on some polls (Poll 1, Poll 4, Poll 5, and Poll 17 had a low participation rate). Thank you to everyone who participated, and to the proposal authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1: Send "[P0876R14]: fiber_context - fibers without scheduler" to Library Working Group for C++26.	4	7	3	1	0	Consensus in favor.
Poll 2: Send "[P0447R26]: Introduction of std::hive to the standard library" to Library Working Group for C++26.	8	4	3	4	2	Weak consensus in favor.
Poll 3: Send "[P2542R7]: views::concat" to Library Working Group for C++26.	11	8	0	0	0	Unanimous consensus in favor.
Poll 4: Send "[P2642R5]: Padded mdspan layouts" to Library Working Group for C++26.	7	8	1	0	0	Strong consensus in favor.
Poll 5: Send "[P2663R5]: Proposal to support interleaved complex values in std::simd" to Library Working Group for C++26.	7	4	1	1	0	Consensus in favor.
Poll 6: Send "[P2810R2]: is_debugger_present is_replaceable" to Library Working Group for C++26.	9	10	2	0	0	Strong consensus in favor.
Poll 7: Send "[P2809R2]: Trivial infinite loops are not Undefined Behavior" to Library Working Group for C++26.	8	11	1	0	0	Strong consensus in favor.
Poll 8: Send "[P2845R5]: Formatting of std::filesystem::path" to Library Working Group for C++26.	8	10	0	1	0	Consensus in favor.
Poll 9: Send "[P2862R1]: text_encoding::name() should never return null values" to Library Working Group for C++26.	9	10	2	0	0	Strong consensus in favor.

Poll 10: Send "[P2867R1]: Remove Deprecated strstreams From C++26" to Library Working Group for C++26.	19	4	0	0	0	Unanimous consensus in favor.
Poll 11: Send "[P2869R3]: Remove Deprecated shared_ptr Atomic Access APIs From C++26" to Library Working Group for C++26.	12	7	0	0	0	Unanimous consensus in favor.
Poll 12: Send "[P2866R1]: Remove Deprecated Volatile Features From C++26" to Library Working Group for C++26.	12	6	3	0	0	Strong consensus in favor.
Poll 13: Send "[P2944R2]: Comparisons for reference_wrapper" to Library Working Group for C++26.	11	10	0	1	0	Consensus in favor.
Poll 14: Send "[P2933R1]: Extend header function with overloads for std::simd" to Library Working Group for C++26.	9	7	0	1	0	Consensus in favor.
Poll 15: Send "[P2976R0]: Freestanding Library: algorithm, numeric, and random" to Library Working Group for C++26.	13	6	1	0	1	Consensus in favor.
Poll 16: Send "[P2968R2]: Make std::ignore a first-class object" to Library Working Group for C++26.	10	6	3	2	0	Consensus in favor.
Poll 17: Send "[P2999R3]: Sender Algorithm Customization" to Library Working Group for C++26.	7	6	1	0	1	Consensus in favor.
Poll 18: Approve "[P2267R1]: Library Evolution Policies" and create SD-9 as described.	10	6	3	1	0	Consensus in favor.
Poll 19: Approve "[P2760R1]: A Plan for C++26 Ranges".	10	7	2	0	0	Strong consensus in favor.

All the polls have consensus in favor, and will be forwarded to LWG.

§ 3. Selected Poll Comments

For some of the comments, small parts were removed to anonymize.

§ 3.1. Poll 1: Send "[P0876R14]: fiber_context - fibers without scheduler" to Library Working Group for C++26.

This is a good example of a facility that belongs in the Standard Library: it cannot be implemented in standard c++, and the existence of boost.context is proof there is demand.

— Strongly Favor

Fibers are an important counterpart to coroutines for writing high-concurrency systems on limited sets of OS threads

— Weakly Favor

I hope it's possible to implement this for all platforms, not just the ones that the authors have access to when developing Boost.Context.

— Weakly Favor

There is very little motivation in the paper as to why this is something the standard needs. Sure it's hard to implement in library, but it's done, and i doubt boost.fiber will stop depending on boost.context for the foreseeable future. Given the lack of library support for coroutines in the standard, the fact that P2300 is still making its way through the standard and that this proposal is utterly useless without a lot of higher level constructs, I am not sure this is the right time for this proposal. Speaking of usefulness over time, there is active work and research into kernel support for lightweight thread, which `_might_` negate most of the motivation for this proposal over the next few years. The back and forth on cancellation over the last few revisions is a bit concerning, that being said this is not a user-facility and it looks usable as an implementation detail of a fiber library satisfying the requirements of P2300, so maybe it's fine.

Ultimately, I haven't been involved enough in the discussion to vote strongly, hence neutral.

— Neutral

We don't have a particular need for this, but I can see how it positively impacts other codebases and will enable authorship of portable fiber-using code.

— Neutral

It's not clear to me that this new abstraction is better enough than `std::thread`, `std::jthread`, coroutines, sender/receiver, or third-party libraries to justify expanding the standard library. The word "experience" (as in "implementation experience" or "usage experience") does not appear in the proposal. Looks like the prior art is Boost.Context (2014), but much has changed since 2014 and the proposed wording doesn't closely match Boost.Context anymore. Could the author persuade Boost.Context to adopt the R14 API? If not, why not?

— Weakly Against

§ 3.2. Poll 2: Send "[P0447R26]: Introduction of `std::hive` to the standard library" to Library Working Group for C++26.

This proposal has been a harder sell than it should have been, due to the fact that users of such containers do not go «the open source way» with their code which obscures (to some) the usefulness of this data structure. Many of our users will appreciate this.

— Strongly Favor

This is a useful container for programs that need stable references, which is common in many scenarios. Even though third party solutions might be better suited for maximum performance having it in the standard will help to create a common interface/vocabulary

— Strongly Favor

This is useful, none of the various objections over the years have enough merit to prevent this being added to the standard.

— Weakly Favor

After a long discussion, I'm convinced that this can be a good addition to the standard lib, for some domains (and as it's "isolated" within a " " header, I don't have concerns about the impact on the rest of the community).

— Weakly Favor

Several engineers in the gaming domain see benefits to this library getting more exposure by being standardized. I'm not opposed to this given the popularity of the library and the diversity of those calling for its standardization.

— Neutral

The wording is in a good state, but I'm not sold on this facility being a great fit for the standard library.

— Neutral

Don't feel that it belongs to C++ standard library. It seems very specific container for the specific use-case. On the other hand, if people think it's necessary and see its value I don't want to create a huge obstacle for that.

— Weakly Against

`hive` is a good container, but it does not belong in the standard library. Adding to the standard library is expensive, using precious committee and implementer effort. Changing things in the standard is hard, so mistakes are expensive to fix, and standard facilities cannot always be updated to reflect the latest practices. Standardizing `hive` would be unwise.

— Strongly Against

I still cannot see a compelling motivation for why this need to live in the STL rather than a library. I do not believe standardization will lead to robust and efficient implementations of the feature over time. We are bound by ABI stabilities. `Vector` is best, use a package manager when you choose from the rest.

— Strongly Against

§ 3.3. Poll 3: Send "[P2542R7]: `views::concat`" to Library Working Group for C++26.

We have our own `concat()` that this could replace.

— Strongly Favor

I look forward to this feature as a way to present all kinds of noncontiguous data structures as a single range.

— Strongly Favor

I think it's a little surprising we don't already have this!

— Weakly Favor

This helps complete the ranges library and has some motivating examples.

— Weakly Favor

§ 3.4. Poll 4: Send "[P2642R5]: Padded mdspan layouts" to Library Working Group for C++26.

This feature lets users express two common cases in one, both of which are important to optimize:

1. overalignment (e.g., for SIMD), and
2. the contiguous access that results from many cases of taking `submdspan` of a `layout_left` or `layout_right` `mdspan`.

Regarding Case (2), the proposed layouts exactly describe what the Fortran and C BLAS Standard interface accepts for rank-2 arrays. The main purpose of these layouts is not for the compiler to optimize, but for users to declare that their data have properties that permit optimization. For example, this would permit algorithm specializations that call into optimized C or Fortran libraries. The paper considers design alternatives, such as a strided layout that would permit encoding any combination of strides at compile time. That layout would be more generally useful, but most users would not need its full generality. When I worry about whether the previous sentence is accurate, I tell myself that LEWG was paying attention to that part of the paper and made an informed decision.

— Strongly Favor

A must-have when programming dense matrices.

— Strongly Favor

Even though it increases the complexity of `mdspan`, I'm convinced that this is useful for some users, and as we have defaults, I don't think it creates a burden.

— Weakly Favor

These are needed in many common applications for `mdspan`. While it adds some complexity into the `mdspan`, it will ultimately reduce the complexity for end users, because they will be able to write their application code uniformly and have it behave optimally in various environments or contexts.

— Weakly Favor

I worry about our span library getting overcomplicated, but I recognize the need for this although my organization doesn't have it.

— Neutral

§ 3.5. Poll 5: Send "[P2663R5]: Proposal to support interleaved complex values in `std::simd`" to Library Working Group for C++26.

Interleaved real/imaginary is the norm for many data structures, so we ought to support it

— Strongly Favor

Industrial-proof practice.

— Strongly Favor

I'm not opposed although my organization doesn't have a need for this.

— Neutral

While the paper and language itself are fine, the FP16 technology mentioned is only 1 year old. This limits both user experience and feedback. It might be better to wait a few years to guarantee that this instruction set will not be amended/up-versioned, before settling on an API. It is also only available in high-end CPUs (for Intel), which means it will not be used in gaming, where it would be of some benefit. As such this paper seems to serve a fraction of a fraction of users.

— Weakly Against

§ 3.6. Poll 6: Send "[P2810R2]: `is_debugger_present` is_replaceable" to Library Working Group for C++26.

Widely useful and widely implemented.

— Strongly Favor

I'm not a big fan of the `std::` debugger functions (they seem like unpleasant to use for beginners compared with existing vendor-provided facilities) but given that we'll get them, this feature should be part of the set.

— Strongly Favor

While I debate the specific need for the original functionality there is no doubt that this would an improvement on it.

— Weakly Favor

I don't need that feature, but people might want to have it.

— Neutral

§ 3.7. Poll 7: Send "[P2809R2]: Trivial infinite loops are not Undefined Behavior" to Library Working Group for C++26.

I support that which weakens the division between C and C++, within reason, and this seems a solid case for where C++ is not quite in the right.

— Strongly Favor

trivial infinite loops are common used in daemon's and business rules engines.

— Strongly Favor

Less trivial UB is better, especially when the programmer's intent is clear.

— Weakly Favor

This makes C++ more consistent with C, and blesses a common idiom in low-level code, without losing the forward progress guarantees that C++ needs to reason about threads. We appreciate how the author conservatively started with matching the C semantics (in R0), then incorporated feedback to develop a more refined model. I don't really like that the specific syntactic form of the loop affects behavior. I'd rather that users had to opt in to a special library function or language construct in order to write an infinite loop. On the other hand, the author's approach blesses common practice and consistency with C. We should favor those two things without strong reasons against.

— Weakly Favor

yield_forever is an okay-ish name.

— Weakly Favor

§ 3.8. Poll 8: Send "[P2845R5]: Formatting of std::filesystem::path" to Library Working Group for C++26.

Being able to format paths is very useful and this paper adds support for this leveraging existing solutions such as escaping and Unicode support in std::format.

— Strongly Favor

This should improve diagnostics.

— Strongly Favor

This functionality will get use in my organization and I expect users in general to make use of this.

— Weakly Favor

§ 3.9. Poll 9: Send "[P2862R1]: `text_encoding::name()` should never return null values" to Library Working Group for C++26.

This improves compatibility with C and makes the interface more consistent.

— Strongly Favor

The billion dollar mistake in other languages is only the hundred dollar feature in C++. Please let's not increase this cost.

— Strongly Favor

This change makes it easier to work with the `text_encoding` API.

— Weakly Favor

`nullptr` checks make client code convoluted

— Weakly Favor

I can honestly see value to both the existing and proposed return values.

— Neutral

This is a bug fix to a rarely used facility. I'm not opposed.

— Neutral

§ 3.10. Poll 10: Send "[P2867R1]: Remove Deprecated strstreams From C++26" to Library Working Group for C++26.

After decades of depreciation, we finally have all replacements in the standard library where stringstream could have claimed to provide superior performance. It no longer has.

— Strongly Favor

stringstream are a terribly broken API, there are much better facilities for this, even in iostreams.

— Strongly Favor

Removing unsafe cruft in iostream is always welcomed

— Strongly Favor

These have been superseded now, at last.

— Weakly Favor

§ 3.11. Poll 11: Send "[P2869R3]: Remove Deprecated shared_ptr Atomic Access APIs From C++26" to Library Working Group for C++26.

These should be removed since they are bug-prone.

— Strongly Favor

If used, will be used wrongly, therefore I'm happy to remove deprecated features and do cleanups.

— Strongly Favor

The API is not in itself problematic, but there are better ways to express things, and we should encourage them.

— Weakly Favor

The rationale in the paper is completely bogus. These APIs have no silent UB trap, that's based on a misunderstanding and has no justification in the standard. But I do agree that atomic > is better, so we should push people to that.

— Weakly Favor

§ 3.12. Poll 12: Send "[P2866R1]: Remove Deprecated Volatile Features From C++26" to Library Working Group for C++26.

I was scared this one would go too far, but reading the details I think it's fine and reasonable.

— Strongly Favor

volatile should only be used where special memory is used.

— Strongly Favor

I'm skeptical of the language changes there, e.g. changing `decltype(vi=42)` from `volatile int&` to `void` so that `std::assignable_from<volatile int&, int>` will now be false not true. But that's not an LEWG concern.

— Weakly Favor

Happy to remove deprecated features and do cleanups, especially if the behaviour is not as expected

— Weakly Favor

It sounds like there are some questions remaining, at least in Mattermost discussion on the SG01 channel. The concern is behavior of interprocess communication via atomics.

— Neutral

It was hard to follow the series of deprecations/undeprecations to understand where volatile was in fact being preserved. A single table for all removed/non-removed areas of it's use would be strongly appreciated.

— Neutral

§ 3.13. Poll 13: Send "[P2944R2]: Comparisons for reference_wrapper" to Library Working Group for C++26.

reference_wrapper should be as close to a T& as possible, and the current inconsistencies are annoying.

— Strongly Favor

This proposal makes reference_wrapper behave more consistently and usefully.

The set of users who use types whose operator== returns non-bool (e.g., valarray, or simd types) probably does not intersect much with the set of users who use reference_wrapper at all. Those in that small intersection almost certainly do not find themselves using reference_wrapper with those types.

— Strongly Favor

As we don't block the ability to support non-trivial types, I'm happy to support this modification.

— Weakly Favor

bool op==, rather than auto op==, seems petty when the cost of specifying auto op== is so low. It doesn't even accomplish the desired goal since other conversions allow op== to (sometimes) work.

— Weakly Against

§ 3.14. Poll 14: Send "[P2933R1]: Extend header function with overloads for `std::simd`" to Library Working Group for C++26.

A straightforward paper that does everything consistently with a header where applicable.

— Strongly Favor

Get the most out of your hardware.

— Strongly Favor

This helps fill out SIMD functionality.

— Weakly Favor

This extension seems like a valid use case that we should support in SIMD, to fit to what users expect

— Weakly Favor

§ 3.15. Poll 15: Send "[P2976R0]: Freestanding Library: algorithm, numeric, and random" to Library Working Group for C++26.

This makes (big parts of) the Library more portable / usable in more contexts.

— Strongly Favor

No objections to that, keeping in mind that parallel algorithms are covered.

— Strongly Favor

Freestanding abilities should be applied equally.

— Weakly Favor

I remain unconvinced that investment in freestanding is worthwhile. a) libc++ doesn't support it, b) libstdc++'s freestanding is rarely used, c) libstdc++ makes its own decisions as to what is in free standing and is happy to ignore what we specify, and d) embedded processors have their own standard libraries (e.g. avr-libstdcpp) that include parts of the standard that their devices support (like chono).

If we want to support these use cases, we should find out what they need and support them instead of busying ourselves with work without impact.

— Strongly Against

§ 3.16. Poll 16: Send "[P2968R2]: Make std::ignore a first-class object" to Library Working Group for C++26.

This makes the existing spec more precise, which would be enough improvement already. Making it usable in more contexts is a nice benefit.

— Strongly Favor

This is vocabulary that helps developers to express their intent more clearly.

— Weakly Favor

I prefer the (void) cast, myself, but I can understand the teachability and self-documenting code arguments.

— Weakly Favor

I disagree with the motivation of this proposal, but I otherwise don't have an objection with the specification change.

— Neutral

Teach the void cast. Or [maybe_unused]. Dragging the stdlib into this seems unnecessary.

— Neutral

I don't agree with the author that the style of syntax proposed makes for simple, unambiguous semantics. In fact, it looks incredibly confusing to newcomers, to whom `std::ignore` is an object which doesn't take on any value, and then you're assigning a function to it, which should have no effect, but it does have some effect? `std::ignore` "An object of unspecified type such that any value can be assigned to it with no effect." Sorry, this is just bad karma. Just change standard practice to use `static_cast`. Done.

— Weakly Against

I would really hate people start to use that style of programming in earnest, and I certainly hope this is not recommend by standards... but apparently that ship has sailed?

— Weakly Against

§ 3.17. Poll 17: Send "P2999R3: Sender Algorithm Customization" to Library Working Group for C++26.

This improves the interface and removes excessive use of `tag_invoke`.

— Strongly Favor

This is a critical fix for senders. Without it, customization is completely broken, especially for `on` and `transfer`.

— Strongly Favor

This is an important facility that enables users to archive portable good to optimal performance in diverse concurrency contexts. While it adds some complexity into the S/R machinery it will ultimately reduce complexity for end user of S/R, because they won't have to search for and implement complicated workarounds for the missing customizations.

— Strongly Favor

It's definitely better to have a lazy customization because senders/receivers adaptors should do the right thing base on the execution context they belong to (that is represented by scheduler).

— Weakly Favor

It seems good, but this proposal is very complex and I have not been part of the team that spent significant time reviewing it so taking a strong stance would be irresponsible.

— Weakly Favor

Do we have sender, yet? A bit like putting the cart before the horse.

— Weakly Favor

This library is overly complex and this proposal makes it even more so. I'm unconvinced that "domains" is a well-defined abstraction, structured bindings should be used as a primary API interface for decomposition, and that there is a need for both bottom-up and top-down optimization.

— Strongly Against

§ 3.18. Poll 18: Approve "[P2267R1]: Library Evolution Policies" and create SD-9 as described.

I strongly support this effort. Too many authors have been subjected to changing whims of our working groups (due to changing memberships of our working groups), at least in the last decade. The table at the very end seems confusing to me but the rest of the paper's worthwhile, and matches what I've been advocating in LEWG (when I was there) since 2016.

— Strongly Favor

A set of policies should reduce time spent reviewing proposals that conform to policies.

— Strongly Favor

We desperately need policies, and this is step 0 for achieving that.

— Strongly Favor

Creating policies as a way to improve committee efficiency is a worthwhile experiment although it remains to be seen if it will succeed. I think the "process of setting a policy" should not be different from adopting any other paper though. I, in particular, dislike the idea of the "one month before adoption" hard rule. Consensus should be sufficient.

— Weakly Favor

This is a good start to codifying Library Evolution practices. Constant work and culture change will be needed to make this effort successful.

— Weakly Favor

I think the following requirement: "A survey of the status quo for this topic, in the wider C++ community. Preferably, this should contain the impact on different domains and industries." may be a lot harder to achieve than is expected - though it would doubtless be necessary sometimes. You may want to soften it to "wider C++ community where necessary or the whole of LEWG where it is not".

— Weakly Favor

Sorry, I was not part of the discussion but I carefully read the paper, thus let me explain my Neutral.

Although I pretty much agree with the direction of the paper I don't feel that all the pros of establishing policies listed in the paper really work in practice. First of all, 5.2 says "A policy is any technical rule or technical guideline...", which to me is contradictory. I think the policy (as it is stated in the paper) is much closer to the rule rather than to a guideline because there is a quite formal process behind. The guideline is softer than a rule (at least, in my head). I understand that it's hard to imagine applying even a guideline without a written paper but still as P2267 says it will be time consuming to introduce a policy and reconsider it in the future if necessary. The real value from policy that I see is "Policies make the standardization process friendly for newcomers". That is 100% true because it allows the newcomers to prepare a more solid proposal from the very beginning, assuming the policy also has a rationale coupled with it. In that case it does save C++ standard committee time. "Policies save time for both authors and the committee" is questionable to me if going beyond a newcomers' case. We previously had the experience when we spent time to introduce a policy and then spent time to cancel it. Doing that also creates the inconsistency in C++ standard library, which makes "Policies create uniformity in users' expectations from the behavior of different parts of the standard library" pro questionable as well. What I would like to see is a list of "common practices" or "guidelines", which give everybody (no matter if the attendee is a newcomer or not) a common way of thinking. Of course, guidelines should also be reviewed and provided with a rationale. And I do agree that if the authors on a new proposal do not apply the guideline it should be explained in this new proposal. The key difference for me between policies and guidelines is the latter are softer and potentially could create less overhead if we are able to find the right balance and establish the right process.

— Neutral

Sounds like bureaucratic capture. I'm leery of the proposed rule "In order to schedule a meeting for re-discussing a policy, there should be new information." What if a policy is made that's bad?

— Weakly Against

§ 3.19. Poll 19: Approve "[P2760R1]: A Plan for C++26 Ranges".

Strongly in Favor assuming that authors will add parallel algorithms with ranges as was discussed in both SG9 and LEWG.

— Strongly Favor

It's good for LEWG to have a public, agreed-upon plan for Ranges development. The table in Section 3 offers encouragement -- it shows many range-v3 features already in C++23, and others in progress. The paper helpfully points out some challenges with defining new features like `cache_last`. I generally agree with the prioritization (Tiers 1, 2, and 3).

— Strongly Favor

It seems reasonable, but ranges are not an easily approachable part of our library these days and it's difficult to forge a strong opinion on these features without spending a `_lot_` of time on them.

— Weakly Favor

Plan seems reasonable. I do hope that things come up that `_aren't_` in the plan though, like a reasonable output range abstraction that doesn't have trivial buffer overflow footguns.

— Weakly Favor

§ References

§ Informative References

[P0447R26]

Matt Bentley. *Introduction of `std::hive` to the standard library*. 17 December 2023. URL: <https://wg21.link/p0447r26>

[P0876R14]

Oliver Kowalke, Nat Goodspeed. *fiber_context - fibers without scheduler*. 13 October 2023. URL: <https://wg21.link/p0876r14>

[P2267R1]

Inbal Levi, Ben Craig, Fabio Fracassi. *Library Evolution Policies*. 23 November 2023. URL: <https://wg21.link/p2267r1>

[P2542R7]

Hui Xie, S. Levent Yilmaz. *views::concat*. 2 December 2023. URL: <https://wg21.link/p2542r7>

[P2642R5]

Christian Trott, Mark Hoemmen, Damien Lebrun-Grandie, Nicolas Morales, Malte Förster, Jiaming Yuan. *Padded `mdspan` layouts*. 5 December 2023. URL: <https://wg21.link/p2642r5>

[P2663R5]

Daniel Towner, Ruslan Arutyunyan. *[Proposal to support interleaved complex values in std::simd](https://wg21.link/p2663r5)*. 8 December 2023. URL: <https://wg21.link/p2663r5>

[P2760R1]

Barry Revzin. *[A Plan for C++26 Ranges](https://wg21.link/p2760r1)*. 15 December 2023. URL: <https://wg21.link/p2760r1>

[P2809R2]

JF Bastien. *[Trivial infinite loops are not Undefined Behavior](https://wg21.link/p2809r2)*. 14 October 2023. URL: <https://wg21.link/p2809r2>

[P2810R2]

René Ferdinand Rivera Morell, Ben Craig. *[is_debugger_present is replaceable](https://wg21.link/p2810r2)*. 3 December 2023. URL: <https://wg21.link/p2810r2>

[P2845R5]

Victor Zverovich. *[Formatting of std::filesystem::path](https://wg21.link/p2845r5)*. 24 November 2023. URL: <https://wg21.link/p2845r5>

[P2862R1]

Daniel Krügler. *[text_encoding::name\(\) should never return null values](https://wg21.link/p2862r1)*. 24 September 2023. URL: <https://wg21.link/p2862r1>

[P2866R1]

Alisdair Meredith. *[Remove Deprecated Volatile Features From C++26](https://wg21.link/p2866r1)*. 16 September 2023. URL: <https://wg21.link/p2866r1>

[P2867R1]

Alisdair Meredith. *[Remove Deprecated strstreams From C++26](https://wg21.link/p2867r1)*. 16 September 2023. URL: <https://wg21.link/p2867r1>

[P2869R3]

Alisdair Meredith. *[Remove Deprecated `shared\\_ptr` Atomic Access APIs From C++26](https://wg21.link/p2869r3)*. 3 December 2023. URL: <https://wg21.link/p2869r3>

[P2933R1]

Daniel Towner, Ruslan Arutyunyan. *[std::simd overloads for <bit> header](https://wg21.link/p2933r1)*. 8 December 2023. URL: <https://wg21.link/p2933r1>

[P2944R2]

Barry Revzin. *[Comparisons for reference_wrapper](https://wg21.link/p2944r2)*. 17 September 2023. URL: <https://wg21.link/p2944r2>

[P2968R2]

Peter Sommerlad. *[Make std::ignore a first-class object](https://wg21.link/p2968r2)*. 13 December 2023. URL: <https://wg21.link/p2968r2>

[P2976R0]

Ben Craig. *Freestanding Library: algorithm, numeric, and random*. 17 September 2023. URL: <https://wg21.link/p2976r0>

[P2999R3]

Eric Niebler. *Sender Algorithm Customization*. 13 December 2023. URL: <https://wg21.link/p2999r3>

[P3053R0]

Inbal Levi, Fabio Fracassi, Ben Craig, Nevin Liber, Billy Baker, Corentin Jabot. *2023-12 Library Evolution Polls*. 15 December 2023. URL: <https://wg21.link/p3053r0>